```
# Connecting this Colab notebook to Kaggle for Maestro Dataset (100GB!)
#!pip install --upgrade kaggle
```

```
# from google.colab import files
# files.upload()
```

```
# !mkdir -p ~/.kaggle
# !cp kaggle.json ~/.kaggle/
# !chmod 600 ~/.kaggle/kaggle.json
```

```
!kaggle --version
```

```
Kaggle CLI 2.0.2
```

Importing Dataset

```
!unzip /content/maestro-v3.0.0-midi.zip -d /content/maestro_midi
```

```
Archive:  /content/maestro-v3.0.0-midi.zip
replace /content/maestro_midi/maestro-v3.0.0/2004/MIDI-Unprocessed_XP_08_R1_2004_01-02_ORIG_MID--AUDIO_08_R1_2004_01_Track01_w
```

```
!pip install pretty_midi
```

```
Requirement already satisfied: pretty_midi in /usr/local/lib/python3.12/dist-packages (0.2.11.post0)
Requirement already satisfied: numpy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from pretty_midi) (2.0.2)
Requirement already satisfied: mido>=1.1.16 in /usr/local/lib/python3.12/dist-packages (from pretty_midi) (1.3.3)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from pretty_midi) (1.17.0)
Requirement already satisfied: importlib_resources in /usr/local/lib/python3.12/dist-packages (from pretty_midi) (7.1.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from mido>=1.1.16->pretty_midi) (26.2)
```

```
import glob
import os

import numpy as np
import pretty_midi
import matplotlib.pyplot as plt
```

```
MIDI_DIR = "/content/maestro_midi/maestro-v3.0.0" # contains year subfolders: 2004, 2008, ...
OUT_DIR = "/content/pairs"
FRAME_RATE = 20        # samples per second
FLATTEN_WINDOW = 4.0   # seconds, smoothing window used to strip phrasing for the raw curve

os.makedirs(OUT_DIR, exist_ok=True)
```

```
from scipy.signal import medfilt
from scipy.ndimage import gaussian_filter1d

## using both median filter and Gaussian filtering for smooth curve

def smooth_cc1(envelope):
    # Step 1: remove jitter
    env_med = medfilt(envelope, kernel_size=9)

    # Step 2: smooth transitions
    env_smooth = gaussian_filter1d(env_med, sigma=2)

    return env_smooth



## Applying file level normalisations instead of Global

def normalize_curve(curve):
    cmin, cmax = curve.min(), curve.max()
    if cmax - cmin < 1e-6:
        return np.zeros_like(curve)
    return (curve - cmin) / (cmax - cmin)
```

```
def seconds_to_frames(seconds, frame_rate):
    return int(seconds * frame_rate)

def make_overlapping_windows(curve, window_size, stride):
    n = len(curve)
    windows = []

    for start in range(0, n - window_size + 1, stride):
```

```
        end = start + window_size
        windows.append(curve[start:end])

    return windows


def window_pair(raw_curve, good_curve, window_size, stride):
    raw_windows  = make_overlapping_windows(raw_curve,  window_size, stride)
    good_windows = make_overlapping_windows(good_curve, window_size, stride)

    # Pair them up
    return list(zip(raw_windows, good_windows))
```

Extraction Functions

```
def extract_velocity_envelope(pm: pretty_midi.PrettyMIDI, frame_times: np.ndarray) -> np.ndarray:
    """CC11 source: continuous dynamics envelope from note velocities."""
    onsets, velocities = [], []
    for instrument in pm.instruments:
        if instrument.is_drum:
            continue
        for note in instrument.notes:
            onsets.append(note.start)
            velocities.append(note.velocity)

    if len(onsets) < 2:
        return np.array([])

    order = np.argsort(onsets)
    onsets = np.array(onsets)[order]
    velocities = np.array(velocities)[order]

    envelope = np.interp(frame_times, onsets, velocities)
    return envelope / 127.0


def extract_sustain_envelope(pm: pretty_midi.PrettyMIDI, frame_times: np.ndarray) -> np.ndarray:
    """CC1 source/proxy: continuous envelope from sustain pedal (CC 64)."""
    times, values = [], []
    for instrument in pm.instruments:
        if instrument.is_drum:
            continue
        for cc in instrument.control_changes:
            if cc.number == 64:
                times.append(cc.time)
                values.append(cc.value)

    if len(times) < 2:
        return np.array([])

    order = np.argsort(times)
    times = np.array(times)[order]
    values = np.array(values)[order]

    envelope = np.interp(frame_times, times, values)
    return envelope / 127.0


def flatten_envelope(envelope: np.ndarray, frame_rate: int, window_seconds: float) -> np.ndarray:
    """Strip phrase-level shaping via a wide moving average -> the 'raw' curve."""
    window = max(int(window_seconds * frame_rate), 1)
    if window >= len(envelope):
        return np.full_like(envelope, envelope.mean())

    kernel = np.ones(window) / window
    flat = np.convolve(envelope, kernel, mode="same")

    flat_min, flat_max = flat.min(), flat.max()
    if flat_max - flat_min > 1e-6:
        flat = (flat - flat_min) / (flat_max - flat_min)
    return flat

def process_file(path: str, frame_rate: int, flatten_window: float):
    pm = pretty_midi.PrettyMIDI(path)
    duration = pm.get_end_time()
    if duration <= 0:
        return None

    all_pairs = []
    n_frames = max(int(duration * frame_rate), 2)
    frame_times = np.linspace(0, duration, n_frames)
```

```
    # 1. Extract
    good_cc11 = extract_velocity_envelope(pm, frame_times)
    good_cc1  = extract_sustain_envelope(pm, frame_times)

    # 2. Smooth CC1 BEFORE normalization
    good_cc1 = smooth_cc1(good_cc1)

    # 3. Normalize AFTER smoothing
    good_cc11 = normalize_curve(good_cc11)
    good_cc1  = normalize_curve(good_cc1)

    if good_cc11.size == 0 and good_cc1.size == 0:
        return None

    result = {}

    # 4. Flatten (raw)
    if good_cc11.size > 0:
        result["good_cc11"] = good_cc11.astype(np.float32)
        result["raw_cc11"]  = flatten_envelope(good_cc11, frame_rate, flatten_window).astype(np.float32)

    if good_cc1.size > 0:
        result["good_cc1"] = good_cc1.astype(np.float32)
        result["raw_cc1"]  = flatten_envelope(good_cc1, frame_rate, flatten_window).astype(np.float32)

    return result
```

```
Start coding or generate with AI.
```

5. Run extraction over the full dataset

This recursively picks up every year subfolder (2004, 2008, ...) automatically. Not every file will have CC1 data — sustain pedal use varies a lot by piece, so `n_missing_cc1` tells you your real CC1 sample size.

```
from tqdm.notebook import tqdm

midi_files = glob.glob(os.path.join(MIDI_DIR, "**", "*.midi"), recursive=True)
midi_files += glob.glob(os.path.join(MIDI_DIR, "**", "*.mid"), recursive=True)
print(f"Found {len(midi_files)} MIDI files")

n_saved = 0
n_missing_cc1 = 0
for path in tqdm(midi_files):
    try:
        result = process_file(path, FRAME_RATE, FLATTEN_WINDOW)
        if result is None:
            continue

        if "good_cc1" not in result:
            n_missing_cc1 += 1

        name = os.path.splitext(os.path.basename(path))[0]
        out_path = os.path.join(OUT_DIR, f"{name}.npz")
        np.savez(out_path, **result)
        n_saved += 1
    except Exception as e:
        print(f"  skipped {path}: {e}")

print(f"Saved {n_saved} files to {OUT_DIR}")
if n_missing_cc1:
    print(f"  ({n_missing_cc1} files had no usable CC1/pedal data -- CC11-only for those)")
```

```
Found 1276 MIDI files
100%                                                1276/1276 [04:35<00:00,  3.01it/s]
/tmp/ipykernel_757/132224264.py:8: UserWarning: kernel_size exceeds volume extent: the volume will be zero-padded.
  env_med = medfilt(envelope, kernel_size=9)
  skipped /content/maestro_midi/maestro-v3.0.0/2015/MIDI-Unprocessed_R1_D2-13-20_mid--AUDIO-from_mp3_20_R1_2015_wav--1.midi: z
  skipped /content/maestro_midi/maestro-v3.0.0/2008/MIDI-Unprocessed_18_R1_2008_01-04_ORIG_MID--AUDIO_18_R1_2008_wav--1.midi:
  skipped /content/maestro_midi/maestro-v3.0.0/2009/MIDI-Unprocessed_06_R1_2009_03-07_ORIG_MID--AUDIO_06_R1_2009_06_R1_2009_04
  skipped /content/maestro_midi/maestro-v3.0.0/2013/ORIG-MIDI_01_7_6_13_Group__MID--AUDIO_04_R1_2013_wav--1.midi: zero-size ar
  skipped /content/maestro_midi/maestro-v3.0.0/2011/MIDI-Unprocessed_20_R1_2011_MID--AUDIO_R1-D8_02_Track02_wav.midi: zero-siz
  skipped /content/maestro_midi/maestro-v3.0.0/2011/MIDI-Unprocessed_05_R1_2011_MID--AUDIO_R1-D2_08_Track08_wav.midi: zero-siz
  skipped /content/maestro_midi/maestro-v3.0.0/2011/MIDI-Unprocessed_07_R1_2011_MID--AUDIO_R1-D3_02_Track02_wav.midi: zero-siz
Saved 1269 files to /content/pairs
```

6. Sanity-check: plot raw vs. good curves for one file

Before building anything else, look at a real example to confirm the curves look right — `good` should show visible swells/accents, `raw` should look flat/smoothed by comparison.

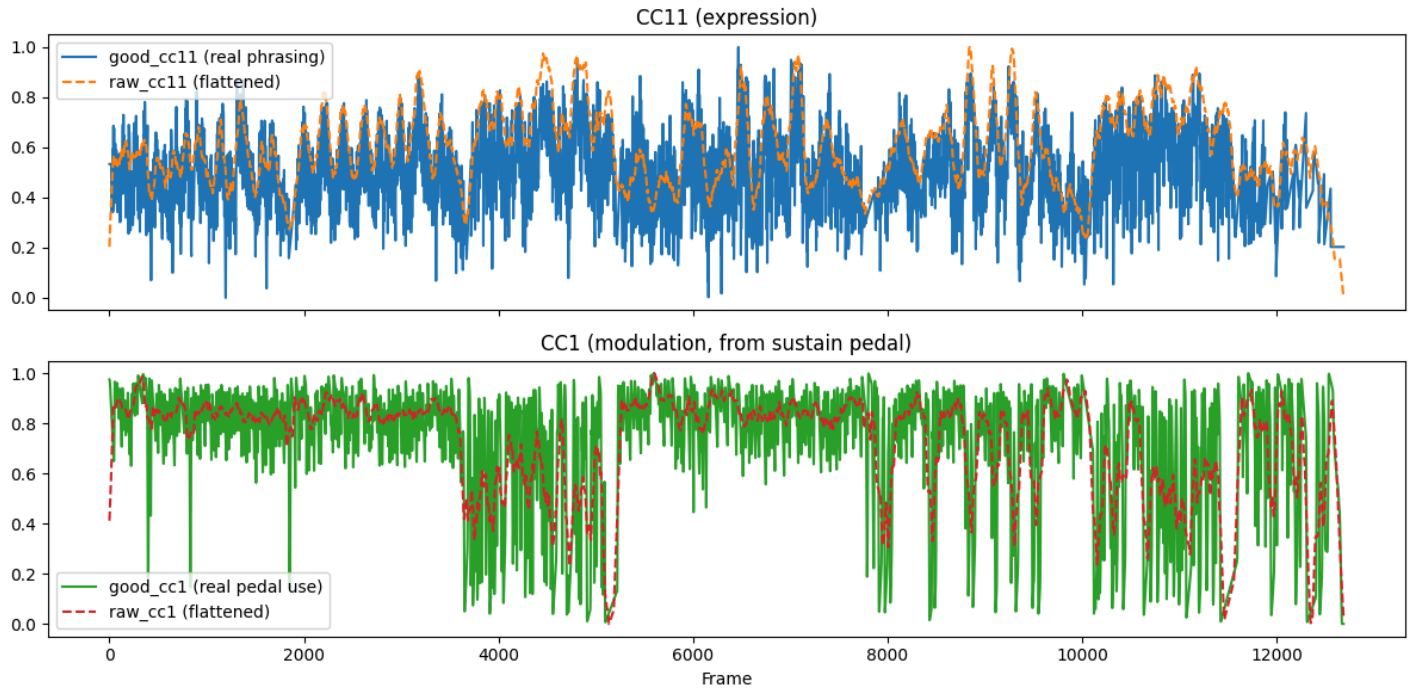
raw_cc11 / raw_cc1 flattened version with phrase-level dynamics removed

good_cc11 / good_cc1 expressive, real musical envelope

We check the training data to see how it looks currently. Think of it like supervised training data (raw:expected good output)

```
sample_files = glob.glob(os.path.join(OUT_DIR, "*.npz"))
print(f"{len(sample_files)} saved pair files available")

d = np.load(sample_files[0])
print("Channels in this file:", d.files)

fig, axes = plt.subplots(2, 1, figsize=(12, 6), sharex=True)

if "good_cc11" in d.files:
    axes[0].plot(d["good_cc11"], label="good_cc11 (real phrasing)", color="tab:blue")
    axes[0].plot(d["raw_cc11"], label="raw_cc11 (flattened)", color="tab:orange", linestyle="--")
    axes[0].set_title("CC11 (expression)")
    axes[0].legend()

if "good_cc1" in d.files:
    axes[1].plot(d["good_cc1"], label="good_cc1 (real pedal use)", color="tab:green")
    axes[1].plot(d["raw_cc1"], label="raw_cc1 (flattened)", color="tab:red", linestyle="--")
    axes[1].set_title("CC1 (modulation, from sustain pedal)")
    axes[1].legend()
else:
    axes[1].set_title("No CC1 data in this file (sustain pedal not used)")

plt.xlabel("Frame")
plt.tight_layout()
plt.show()
```

```
1269 saved pair files available
Channels in this file: ['good_cc11', 'raw_cc11', 'good_cc1', 'raw_cc1']
```



Double-click (or enter) to edit

```
Start coding or generate with AI.
```

Improvements done:::

1️⃣ Smooth CC64 (pedal) before flattening The green CC1 curve in your chart is too jittery. Sustain pedal data often flickers rapidly due to half-pedaling.

A simple median filter or low-pass filter will dramatically improve the target curve:

Removes noise

Preserves musical intent

Makes the supervised mapping easier

```
Start coding or generate with AI.
```

7. Build the training set with overlapping windows

Slide a 4 s window (80 frames at 20 FPS) across each saved pair with a 1 s stride (20 frames) for 75% overlap — more, well-covered samples from the same files. `window_pair` reuses the existing helper: raw window in, good window out.

```
DATA_DIR = "/content/dataset"
os.makedirs(DATA_DIR, exist_ok=True)

WINDOW_FRAMES = seconds_to_frames(4.0, FRAME_RATE)   # 80 -> matches FLATTEN_WINDOW
STRIDE        = seconds_to_frames(1.0, FRAME_RATE)   # 20 -> 75% overlap
print(f"[cfg] window_frames={WINDOW_FRAMES}, stride={STRIDE} "
      f"(each window covers 4.0s, step 1.0s)")
print(f"[cfg] one sample (x or y) will have shape ({WINDOW_FRAMES},) -> {WINDOW_FRAMES} float32 values")
print(f"[cfg] model input per channel = (n_samples, {WINDOW_FRAMES}); "
      f"add a trailing axis for LSTM/Conv1D -> (n_samples, {WINDOW_FRAMES}, 1)")

# --- 1) SPLIT BY FILE (leakage-free: no song appears in two splits) ---
rng = np.random.default_rng(42)
pair_files = sorted(glob.glob(os.path.join(OUT_DIR, "*.npz")))
rng.shuffle(pair_files)
n = len(pair_files)
n_val = int(0.10 * n)
n_test = int(0.10 * n)
splits = {
    "train": pair_files[:n - n_val - n_test],
    "val":   pair_files[n - n_val - n_test:n - n_test],
    "test":  pair_files[n - n_test:],
}
for name, files in splits.items():
    print(f"[split] {name}: {len(files)} files")

def build_windows(files, raw_key, good_key, tag):
    """Build X/y windows + per-window file provenance for one split and one channel."""
    X, y, file_idx = [], [], []
    too_short = 0
    missing = 0
    for fi, f in enumerate(files):
        d = np.load(f)
        if raw_key not in d.files or good_key not in d.files:
            missing += 1
            print(f"[warn][{tag}] {os.path.basename(f)}: missing {raw_key}/{good_key}")
            continue
        if len(d[raw_key]) < WINDOW_FRAMES:
            too_short += 1
            continue
        for x, t in window_pair(d[raw_key], d[good_key], WINDOW_FRAMES, STRIDE):
            X.append(x)
            y.append(t)
            file_idx.append(fi)
    X = np.asarray(X, dtype=np.float32)
    y = np.asarray(y, dtype=np.float32)
    file_idx = np.asarray(file_idx, dtype=np.int64)
    return X, y, file_idx, too_short, missing

# --- 2) WINDOW EACH SPLIT AND SAVE (CC11 and CC1, one model per channel) ---
for split_name, files in splits.items():
    for ch, (raw_key, good_key) in {
            "cc11": ("raw_cc11", "good_cc11"),
            "cc1":  ("raw_cc1",  "good_cc1"),
    }.items():
        X, y, file_idx, too_short, missing = build_windows(files, raw_key, good_key, f"{split_name}/{ch}")
        out_path = os.path.join(DATA_DIR, f"windows_{ch}_{split_name}.npz")
        np.savez(out_path, X=X, y=y, file_idx=file_idx)
        print(f"[save] {os.path.basename(out_path)}: "
              f"X={X.shape} {X.dtype}, y={y.shape} {y.dtype}, "
              f"{len(files)} files, {too_short} too_short, {missing} missing")

print(f"[done] Saved train/val/test windows to {DATA_DIR}")
```

```
[cfg] window_frames=80, stride=20 (each window covers 4.0s, step 1.0s)
[cfg] one sample (x or y) will have shape (80,) -> 80 float32 values
[cfg] model input per channel = (n_samples, 80); add a trailing axis for LSTM/Conv1D -> (n_samples, 80, 1)
[split] train: 1017 files
[split] val: 126 files
[split] test: 126 files
[save] windows_cc11_train.npz: X=(572725, 80) float32, y=(572725, 80) float32, 1017 files, 0 too_short, 0 missing
[save] windows_cc1_train.npz: X=(572725, 80) float32, y=(572725, 80) float32, 1017 files, 0 too_short, 0 missing
[save] windows_cc11_val.npz: X=(64214, 80) float32, y=(64214, 80) float32, 126 files, 0 too_short, 0 missing
```

```
[save] windows_cc1_val.npz: X=(64214, 80) float32, y=(64214, 80) float32, 126 files, 0 too_short, 0 missing
[save] windows_cc11_test.npz: X=(73538, 80) float32, y=(73538, 80) float32, 126 files, 0 too_short, 0 missing
[save] windows_cc1_test.npz: X=(73538, 80) float32, y=(73538, 80) float32, 126 files, 0 too_short, 0 missing
[done] Saved train/val/test windows to /content/dataset
```

LSTM Model Training

Two things to remember:

1. We Only are training CC11 and CC1 from Maestro dataset
2. We are training CC11 anf CC1 separately through different models so that their learnings do not overlap.

```
# import torch
# import torch.nn as nn
# from torch.utils.data import DataLoader, TensorDataset

# DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# CHANNEL = "cc11"   # one model per channel: set to "cc11" or "cc1" and re-run
# print(f"[cfg] device={DEVICE}, channel={CHANNEL}")

# def load_split(split_name):
#     p = os.path.join(DATA_DIR, f"windows_{CHANNEL}_{split_name}.npz")
#     d = np.load(p)
#     X = torch.tensor(d["X"], dtype=torch.float32).unsqueeze(-1)   # (n, 80) -> (n, 80, 1)
#     y = torch.tensor(d["y"], dtype=torch.float32)                 # (n, 80)
#     print(f"[load] {split_name}: X={tuple(X.shape)} y={tuple(y.shape)}")
#     return X, y

# X_train, y_train = load_split("train")
# X_val,   y_val   = load_split("val")
# X_test,  y_test  = load_split("test")

# BATCH_SIZE = 512
# train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=BATCH_SIZE, shuffle=True)
# val_loader   = DataLoader(TensorDataset(X_val,   y_val),   batch_size=BATCH_SIZE, shuffle=False)
# test_loader  = DataLoader(TensorDataset(X_test,  y_test),  batch_size=BATCH_SIZE, shuffle=False)

# class CurveLSTM(nn.Module):
#     """Seq2seq: maps a 1-D curve (n, seq, 1) to a curve (n, seq)."""
#     def __init__(self, input_size=1, hidden_size=64, num_layers=2):
#         super().__init__()
#         self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
#         self.head = nn.Linear(hidden_size, 1)

#     def forward(self, x):
#         out, _ = self.lstm(x)              # (n, seq, hidden)
#         return self.head(out).squeeze(-1)  # (n, seq)

# model = CurveLSTM().to(DEVICE)
# optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
# criterion = nn.MSELoss()
# n_params = sum(p.numel() for p in model.parameters())
# print(f"[model] {model.__class__.__name__}, params={n_params:,}")

# EPOCHS = 20

# def evaluate(loader):
#     model.eval()
#     total, count = 0.0, 0
#     with torch.no_grad():
#         for xb, yb in loader:
#             pred = model(xb.to(DEVICE))
#             total += criterion(pred, yb.to(DEVICE)).item() * xb.size(0)
#             count += xb.size(0)
#     return total / count

# for epoch in range(1, EPOCHS + 1):
#     model.train()
#     running, count = 0.0, 0
#     for xb, yb in train_loader:
#         xb, yb = xb.to(DEVICE), yb.to(DEVICE)
#         optimizer.zero_grad()
#         loss = criterion(model(xb), yb)
#         loss.backward()
#         optimizer.step()
#         running += loss.item() * xb.size(0)
#         count += xb.size(0)
#     val_mse = evaluate(val_loader)
#     print(f"[epoch {epoch:3d}] train_mse={running/count:.6f}  val_mse={val_mse:.6f}")

# test_mse = evaluate(test_loader)
# print(f"[test ] mse={test_mse:.6f} (baseline: predict-mean -> {((y_test - y_test.mean()).pow(2).mean().item()):.6f})")

# torch.save(model.state_dict(), os.path.join(DATA_DIR, f"curve_lstm_{CHANNEL}.pt"))
```

```
# print(f"[save] weights -> {DATA_DIR}/curve_lstm_{CHANNEL}.pt")

# # quick visual check: predicted vs target for 3 test samples
# import matplotlib.pyplot as plt
# model.eval()
# with torch.no_grad():
#     preds = model(X_test[:3].to(DEVICE)).cpu()
# fig, axes = plt.subplots(1, 3, figsize=(15, 4))
# for i, ax in enumerate(axes):
#     ax.plot(X_test[i].numpy(), label="raw (input)", alpha=0.5)
#     ax.plot(y_test[i].numpy(), label="good (target)")
#     ax.plot(preds[i].numpy(), label="predicted", ls="--")
#     ax.legend(fontsize=8)
#     ax.set_title(f"test sample {i}")
# plt.tight_layout()
# plt.show()
```

Next model: Transformers + Attention Self-attention lets every output frame look at every input frame directly, not just through a compressed sequential hidden state. That's exactly what you need to catch a sharp jump at frame 44 without it getting averaged away by frames 1-43.

Residual/skip connection from raw input to output — the model predicts a correction to add to raw, not the whole curve from scratch. This is a much easier learning problem and tends to preserve sharp local structure instead of "regressing to smooth."

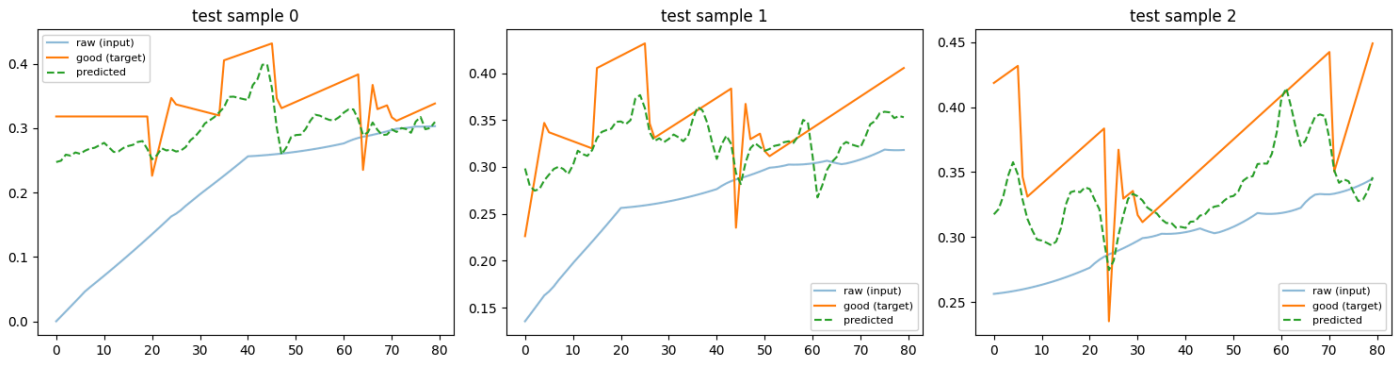
Huber loss instead of MSE — MSE's squared penalty is what was pushing your LSTM toward safe, smooth averages. Huber (SmoothL1) is more forgiving of the model "confidently guessing wrong location" on a spike, so it doesn't get punished into blandness the same way.

```
import os
import math
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
CHANNEL = "cc11"
print(f"[cfg] device={DEVICE}, channel={CHANNEL}")

def load_split(split_name):
    p = os.path.join(DATA_DIR, f"windows_{CHANNEL}_{split_name}.npz")
    d = np.load(p)
    X = torch.tensor(d["X"], dtype=torch.float32).unsqueeze(-1)
    y = torch.tensor(d["y"], dtype=torch.float32)
    print(f"[load] {split_name}: X={tuple(X.shape)} y={tuple(y.shape)}")
    return X, y

X_train, y_train = load_split("train")
X_val,   y_val   = load_split("val")
X_test,  y_test  = load_split("test")

BATCH_SIZE = 512
train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=BATCH_SIZE, shuffle=True)
val_loader   = DataLoader(TensorDataset(X_val,   y_val),   batch_size=BATCH_SIZE, shuffle=False)
test_loader  = DataLoader(TensorDataset(X_test,  y_test),  batch_size=BATCH_SIZE, shuffle=False)

class SinusoidalPositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=80):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float32).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe.unsqueeze(0))

    def forward(self, x):
        return x + self.pe[:, :x.size(1), :]


class CurveTransformer(nn.Module):
    def __init__(self, input_size=1, d_model=64, nhead=4, num_layers=3,
                 dim_feedforward=128, dropout=0.1, seq_len=80):
        super().__init__()
        self.input_proj = nn.Linear(input_size, d_model)
        self.pos_enc = SinusoidalPositionalEncoding(d_model, max_len=seq_len)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead, dim_feedforward=dim_feedforward,
            dropout=dropout, batch_first=True,
        )
        self.encoder = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
        self.head = nn.Linear(d_model, 1)

    def forward(self, x):
        raw = x.squeeze(-1)
```

```
        h = self.input_proj(x)
        h = self.pos_enc(h)
        h = self.encoder(h)
        correction = self.head(h).squeeze(-1)
        return raw + correction


model = CurveTransformer().to(DEVICE)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.SmoothL1Loss(beta=0.05)
mse_for_reporting = nn.MSELoss()
n_params = sum(p.numel() for p in model.parameters())
print(f"[model] {model.__class__.__name__}, params={n_params:,}")

EPOCHS = 100
PATIENCE = 10
best_val_loss = float("inf")
best_epoch = 0
best_state = None
epochs_no_improve = 0

CHECKPOINT_PATH = os.path.join(DATA_DIR, f"curve_transformer_{CHANNEL}_checkpoint.pt")  # <-- persists to disk

def evaluate(loader):
    model.eval()
    total_loss, total_mse, count = 0.0, 0.0, 0
    with torch.no_grad():
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            pred = model(xb)
            total_loss += criterion(pred, yb).item() * xb.size(0)
            total_mse  += mse_for_reporting(pred, yb).item() * xb.size(0)
            count += xb.size(0)
    return total_loss / count, total_mse / count

for epoch in range(1, EPOCHS + 1):
    model.train()
    running, count = 0.0, 0
    for xb, yb in train_loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
        running += loss.item() * xb.size(0)
        count += xb.size(0)

    val_loss, val_mse = evaluate(val_loader)
    train_loss = running / count
    marker = ""

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        best_epoch = epoch
        best_state = {k: v.clone() for k, v in model.state_dict().items()}
        epochs_no_improve = 0
        marker = "  <- best so far"

        torch.save({                        # <-- write to disk immediately, not just RAM
            "model_state": best_state,
            "epoch": epoch,
            "val_huber": val_loss,
            "val_mse": val_mse,
            "optimizer_state": optimizer.state_dict(),
        }, CHECKPOINT_PATH)
    else:
        epochs_no_improve += 1

    print(f"[epoch {epoch:3d}] train_huber={train_loss:.6f}  val_huber={val_loss:.6f}  val_mse={val_mse:.6f}{marker}")

    if epochs_no_improve >= PATIENCE:
        print(f"[stop] no val_huber improvement for {PATIENCE} epochs, stopping at epoch {epoch} "
              f"(best was epoch {best_epoch}, val_huber={best_val_loss:.6f})")
        break

model.load_state_dict(best_state)
print(f"[restore] using weights from epoch {best_epoch}, val_huber={best_val_loss:.6f}")

test_loss, test_mse = evaluate(test_loader)
baseline_mse = ((y_test - y_test.mean()).pow(2).mean().item())
print(f"[test ] huber={test_loss:.6f}  mse={test_mse:.6f} (baseline predict-mean mse -> {baseline_mse:.6f})")

torch.save(model.state_dict(), os.path.join(DATA_DIR, f"curve_transformer_{CHANNEL}.pt"))
```

```
print(f"[save] weights -> {DATA_DIR}/curve_transformer_{CHANNEL}.pt")

import matplotlib.pyplot as plt
model.eval()
with torch.no_grad():
    preds = model(X_test[:3].to(DEVICE)).cpu()
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for i, ax in enumerate(axes):
    ax.plot(X_test[i].numpy(), label="raw (input)", alpha=0.5)
    ax.plot(y_test[i].numpy(), label="good (target)")
    ax.plot(preds[i].numpy(), label="predicted", ls="--")
    ax.legend(fontsize=8)
    ax.set_title(f"test sample {i}")
plt.tight_layout()
plt.show()
```

```
[cfg] device=cuda, channel=cc11
[load] train: X=(572725, 80, 1) y=(572725, 80)
[load] val: X=(64214, 80, 1) y=(64214, 80)
[load] test: X=(73538, 80, 1) y=(73538, 80)
[model] CurveTransformer, params=100,609
[epoch   1] train_huber=0.064108  val_huber=0.059845  val_mse=0.011249  <- best so far
[epoch   2] train_huber=0.056599  val_huber=0.056883  val_mse=0.010429  <- best so far
[epoch   3] train_huber=0.054491  val_huber=0.054677  val_mse=0.009852  <- best so far
[epoch   4] train_huber=0.053198  val_huber=0.053987  val_mse=0.009630  <- best so far
[epoch   5] train_huber=0.052225  val_huber=0.053766  val_mse=0.009509  <- best so far
[epoch   6] train_huber=0.051448  val_huber=0.052249  val_mse=0.009137  <- best so far
[epoch   7] train_huber=0.050819  val_huber=0.051754  val_mse=0.008989  <- best so far
[epoch   8] train_huber=0.050244  val_huber=0.050590  val_mse=0.008714  <- best so far
[epoch   9] train_huber=0.049686  val_huber=0.050568  val_mse=0.008641  <- best so far
[epoch  10] train_huber=0.049204  val_huber=0.049988  val_mse=0.008499  <- best so far
[epoch  11] train_huber=0.048768  val_huber=0.049321  val_mse=0.008332  <- best so far
[epoch  12] train_huber=0.048358  val_huber=0.048909  val_mse=0.008216  <- best so far
[epoch  13] train_huber=0.048035  val_huber=0.049432  val_mse=0.008278
[epoch  14] train_huber=0.047737  val_huber=0.048252  val_mse=0.008044  <- best so far
[epoch  15] train_huber=0.047473  val_huber=0.047899  val_mse=0.007968  <- best so far
[epoch  16] train_huber=0.047241  val_huber=0.047794  val_mse=0.007910  <- best so far
[epoch  17] train_huber=0.047042  val_huber=0.047440  val_mse=0.007839  <- best so far
[epoch  18] train_huber=0.046887  val_huber=0.047550  val_mse=0.007821
[epoch  19] train_huber=0.046709  val_huber=0.047977  val_mse=0.007883
[epoch  20] train_huber=0.046583  val_huber=0.047454  val_mse=0.007760
[epoch  21] train_huber=0.046454  val_huber=0.047450  val_mse=0.007791
[epoch  22] train_huber=0.046335  val_huber=0.047262  val_mse=0.007726  <- best so far
[epoch  23] train_huber=0.046222  val_huber=0.048424  val_mse=0.007949
[epoch  24] train_huber=0.046127  val_huber=0.046979  val_mse=0.007656  <- best so far
[epoch  25] train_huber=0.046034  val_huber=0.047628  val_mse=0.007765
[epoch  26] train_huber=0.045942  val_huber=0.046956  val_mse=0.007623  <- best so far
[epoch  27] train_huber=0.045863  val_huber=0.046732  val_mse=0.007599  <- best so far
[epoch  28] train_huber=0.045771  val_huber=0.046663  val_mse=0.007565  <- best so far
[epoch  29] train_huber=0.045720  val_huber=0.048217  val_mse=0.007882
[epoch  30] train_huber=0.045664  val_huber=0.046446  val_mse=0.007521  <- best so far
[epoch  31] train_huber=0.045580  val_huber=0.047117  val_mse=0.007650
[epoch  32] train_huber=0.045523  val_huber=0.046546  val_mse=0.007533
[epoch  33] train_huber=0.045470  val_huber=0.046648  val_mse=0.007541
[epoch  34] train_huber=0.045416  val_huber=0.046825  val_mse=0.007580
[epoch  35] train_huber=0.045359  val_huber=0.046313  val_mse=0.007490  <- best so far
[epoch  36] train_huber=0.045319  val_huber=0.046650  val_mse=0.007556
[epoch  37] train_huber=0.045280  val_huber=0.046951  val_mse=0.007611
[epoch  38] train_huber=0.045226  val_huber=0.046362  val_mse=0.007488
[epoch  39] train_huber=0.045191  val_huber=0.046947  val_mse=0.007590
[epoch  40] train_huber=0.045165  val_huber=0.046323  val_mse=0.007498
[epoch  41] train_huber=0.045104  val_huber=0.046274  val_mse=0.007469  <- best so far
[epoch  42] train_huber=0.045071  val_huber=0.046219  val_mse=0.007457  <- best so far
[epoch  43] train_huber=0.045033  val_huber=0.046639  val_mse=0.007529
[epoch  44] train_huber=0.045008  val_huber=0.046538  val_mse=0.007505
[epoch  45] train_huber=0.044986  val_huber=0.046141  val_mse=0.007446  <- best so far
[epoch  46] train_huber=0.044947  val_huber=0.046539  val_mse=0.007516
[epoch  47] train_huber=0.044920  val_huber=0.045813  val_mse=0.007363  <- best so far
[epoch  48] train_huber=0.044883  val_huber=0.046788  val_mse=0.007558
[epoch  49] train_huber=0.044852  val_huber=0.046480  val_mse=0.007518
[epoch  50] train_huber=0.044841  val_huber=0.046739  val_mse=0.007552
[epoch  51] train_huber=0.044808  val_huber=0.046531  val_mse=0.007511
[epoch  52] train_huber=0.044776  val_huber=0.046442  val_mse=0.007487
[epoch  53] train_huber=0.044763  val_huber=0.046564  val_mse=0.007505
[epoch  54] train_huber=0.044723  val_huber=0.046121  val_mse=0.007463
[epoch  55] train_huber=0.044717  val_huber=0.045874  val_mse=0.007375
[epoch  56] train_huber=0.044690  val_huber=0.046524  val_mse=0.007546
[epoch  57] train_huber=0.044659  val_huber=0.046768  val_mse=0.007539
[stop] no val_huber improvement for 10 epochs, stopping at epoch 57 (best was epoch 47, val_huber=0.045813)
[restore] using weights from epoch 47, val_huber=0.045813
[test ] huber=0.044818  mse=0.007184 (baseline predict-mean mse -> 0.032419)
[save] weights -> /content/dataset/curve_transformer_cc11.pt
```



Start coding or generate with AI.

Double-click (or enter) to edit

Model test MSE vs baseline (0.032419)

LSTM 0.009582 3.4x better Transformer 0.007475 4.3x better

Model test MSE vs baseline

LSTM (20 epochs, no early stopping) 0.009582 3.4x

Transformer (20 epochs, no early stopping) 0.007475 4.3x

Transformer (early stopping, best=epoch 28) 0.007150 4.5x